EE4705 Project 2 · Group 9

Collision-free path planning with STOMP

Kinova Gen3 and Franka Emika Panda, in MATLAB

Joshua Lim · Hu Jiamo · Dexter

Five tasks, one code base

Task	What we did	Result
1	Completed STOMP on the Kinova Gen3	Collision-free in 23 iterations
2	Moved to the Franka Emika Panda	Collision-free in 13 iterations
3	Product-of-Exponentials kinematics	Same path, 14× faster iterations
4	Three custom multi-obstacle scenes	30 of 30 seeded runs collision-free
5	Kept the gripper’s y-axis upright	Maximum tilt 36.8° → 1.9°

Every result is checked against the robot meshes, at the waypoints and between them.

STOMP: sample, score, weight, update

1 Sample

20 noisy trajectories, smooth noise from R^{-1}

2 Score

Obstacle cost at every waypoint

3 Weight

Cheaper samples get higher probability

4 Update

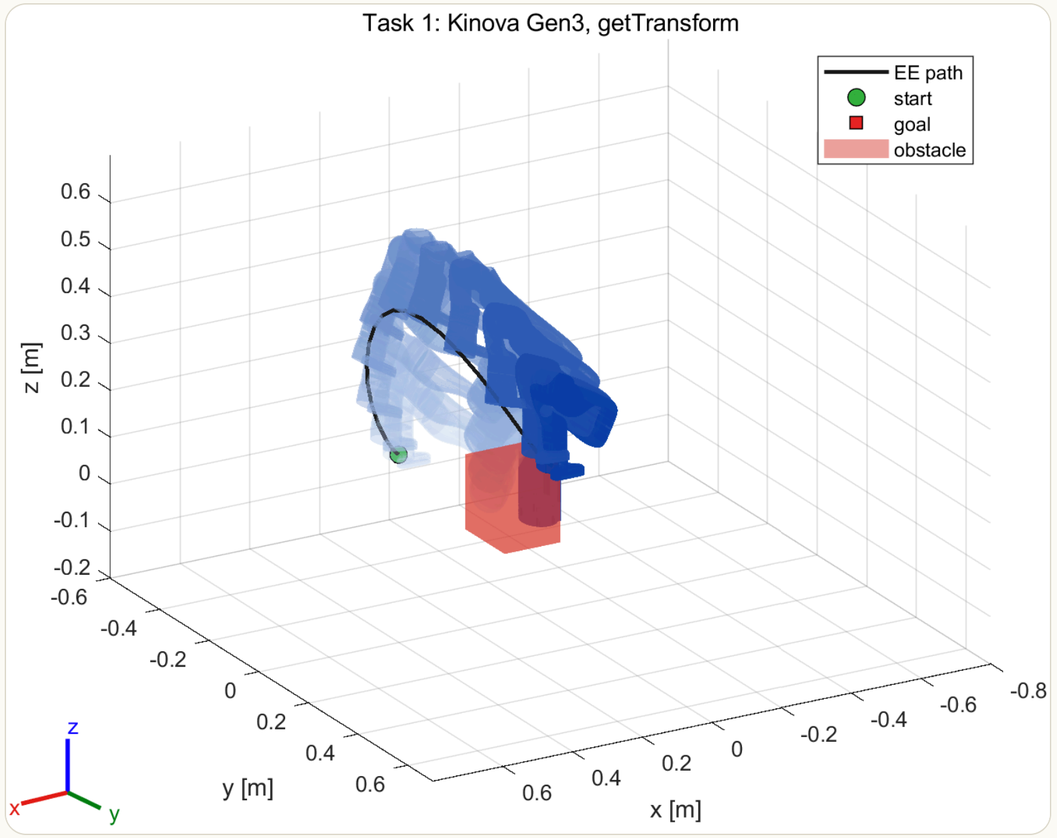
Move θ by the weighted noise, smoothed by M

Obstacle cost, summed over the collision spheres

$$q_o = \sum_b \max\left(\varepsilon + r_b - d(x_b),\ 0\right)\ \|\dot{x}_b\|$$

d comes from a signed distance field of a 2 cm voxel world. No gradient of the cost is needed.

Task 1: Kinova Gen3 around the box



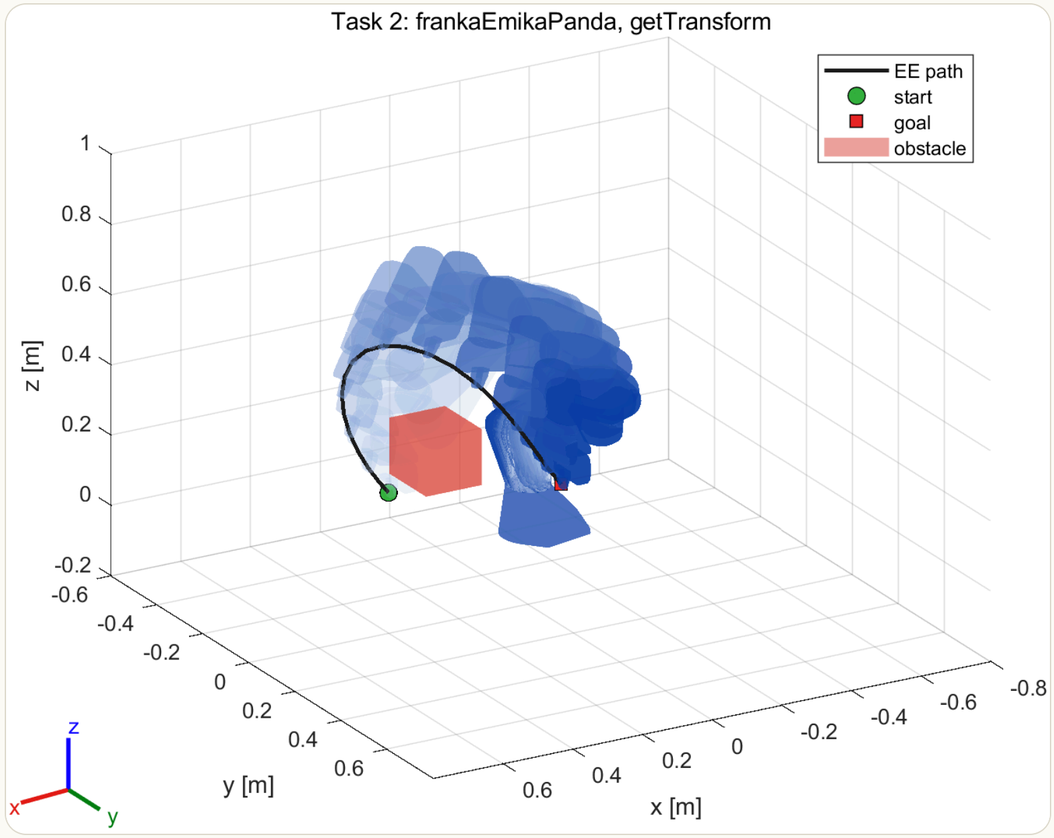
23 iterations

8 of 20 waypoints collided at the start; none at the end

Bugs we fixed in the sample code

- Samples leaving the world scored zero cost
- Control cost added cross-joint terms, 45% too high
- The last ~6 cm of the arm had no spheres

Task 2: Franka Emika Panda



A harder arm

7 joints with tight limits, link offsets, a two-finger gripper

Tool point at the fingertips

Fingers frozen; a TCP frame added 0.1034 m past the flange

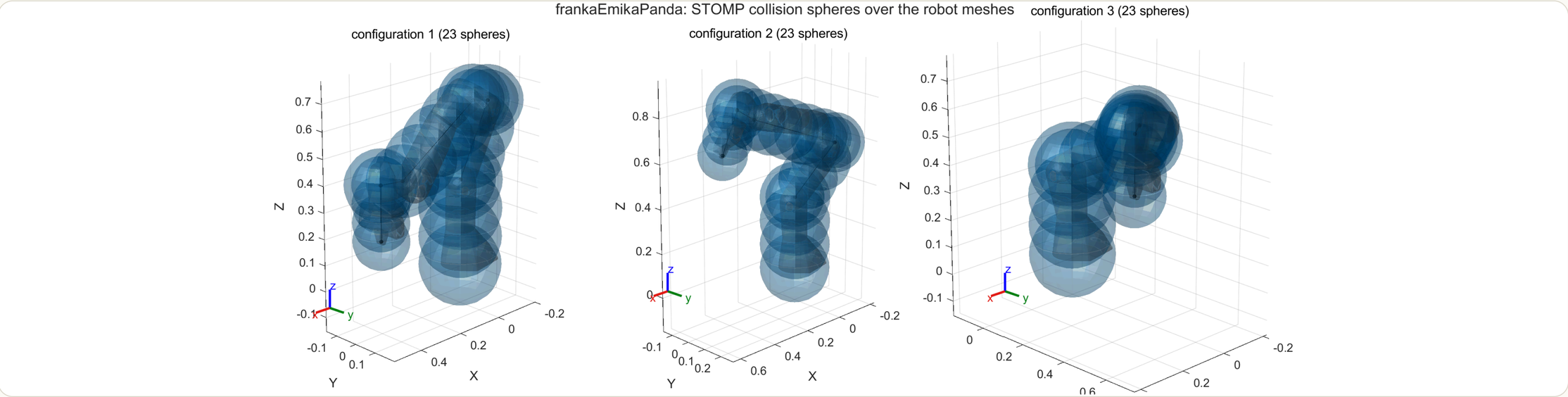
Random restarts for IK

Without them the goal was missed by 8.9 cm

A scene that really collides

Box on the joint-space path: 7 of 20 waypoints; solved in 13 iterations

Collision spheres fitted to the meshes



5 → 10–15 cm
sphere radius, one per link

99%
of mesh vertices inside the spheres

5 → 2 cm
safety margin, now a real margin

One small cost change is not convergence

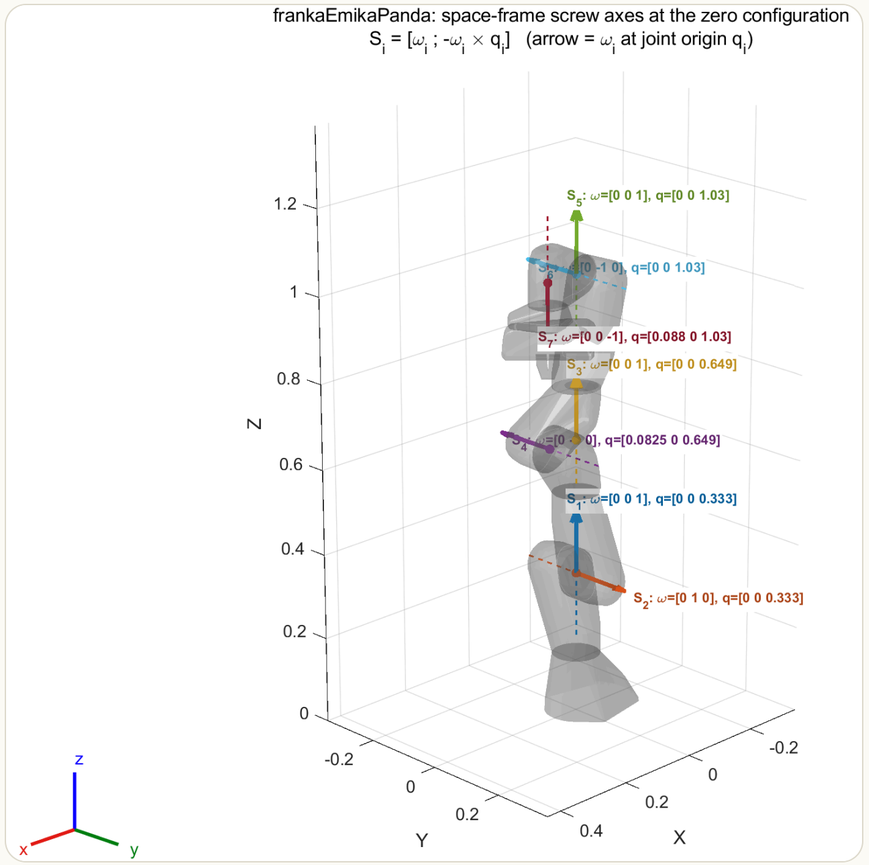
29 → 35

collision-free runs out of 35, going from the original stop rule to three consecutive small changes

Scene	P = 1	P = 3
Task 4 S3	1 / 5	5 / 5
Task 4 S1	4 / 5	5 / 5
Task 5 constrained	4 / 5	5 / 5
Other four scenes	5 / 5	5 / 5

The price is more iterations: 9% more at best, about 6× in the constrained case.

Task 3: screw axes from the robot model



$$T_k(\theta) = e^{[S_1]\theta_1}\, e^{[S_2]\theta_2} \cdots e^{[S_k]\theta_k}\, M_k$$

ω: joint axis in the base frame

The URDF axis is in the joint's own frame, so it must be rotated

$$v_i = -\omega_i \times q_i$$

q is the joint origin at the zero configuration

Checked with the matrix log

Both methods agree to 4e-16; computed once per robot

Task 3: the same answer, much faster

1e-15

largest difference from getTransform, 500 random poses

35×

faster forward kinematics for all frames (Panda)

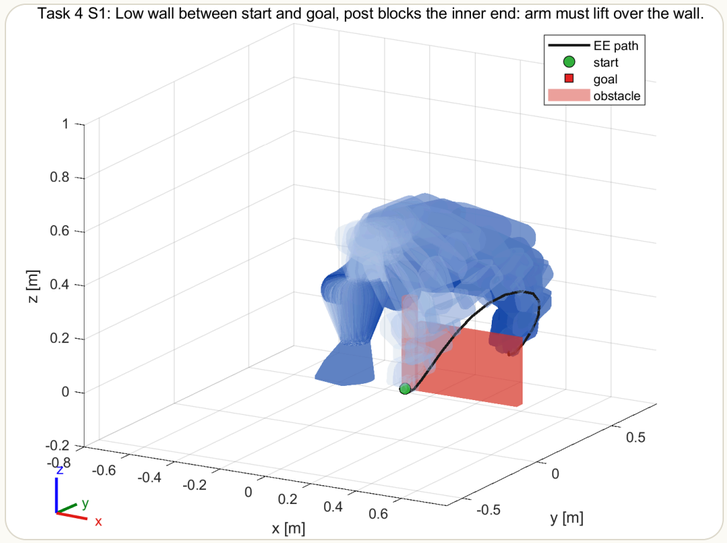
14×

faster STOMP iterations: 527 ms → 38 ms

Why it is fast: one running product gives every frame, with closed-form exponentials instead of expm.

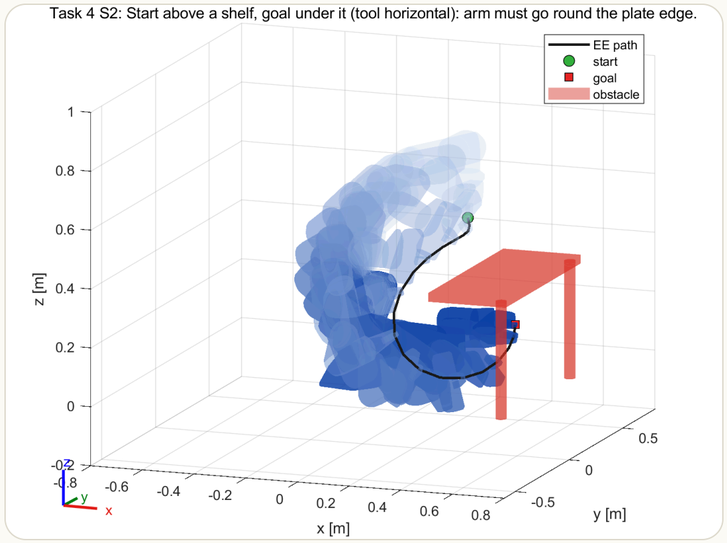
Same seed, same plan: PoE and getTransform give an identical trajectory.

Task 4: three scenes that force a detour



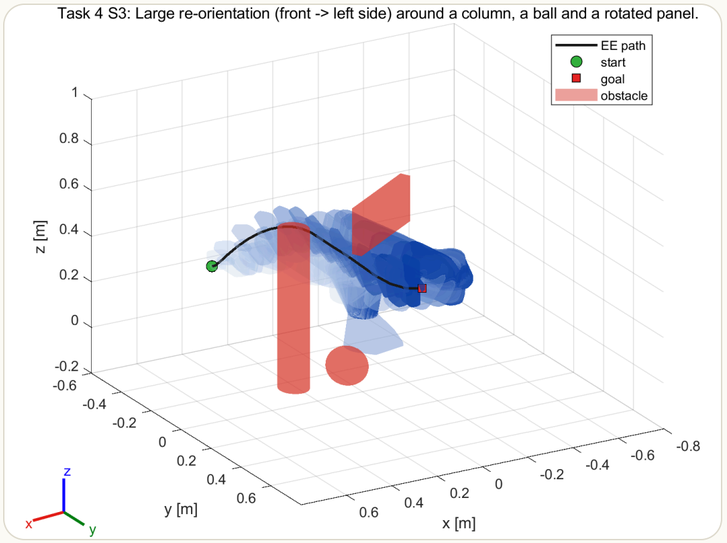
Over the wall

10 of 20 waypoints collide at the start; 32 iterations



Under the shelf

6 of 20 waypoints collide at the start; 20 iterations

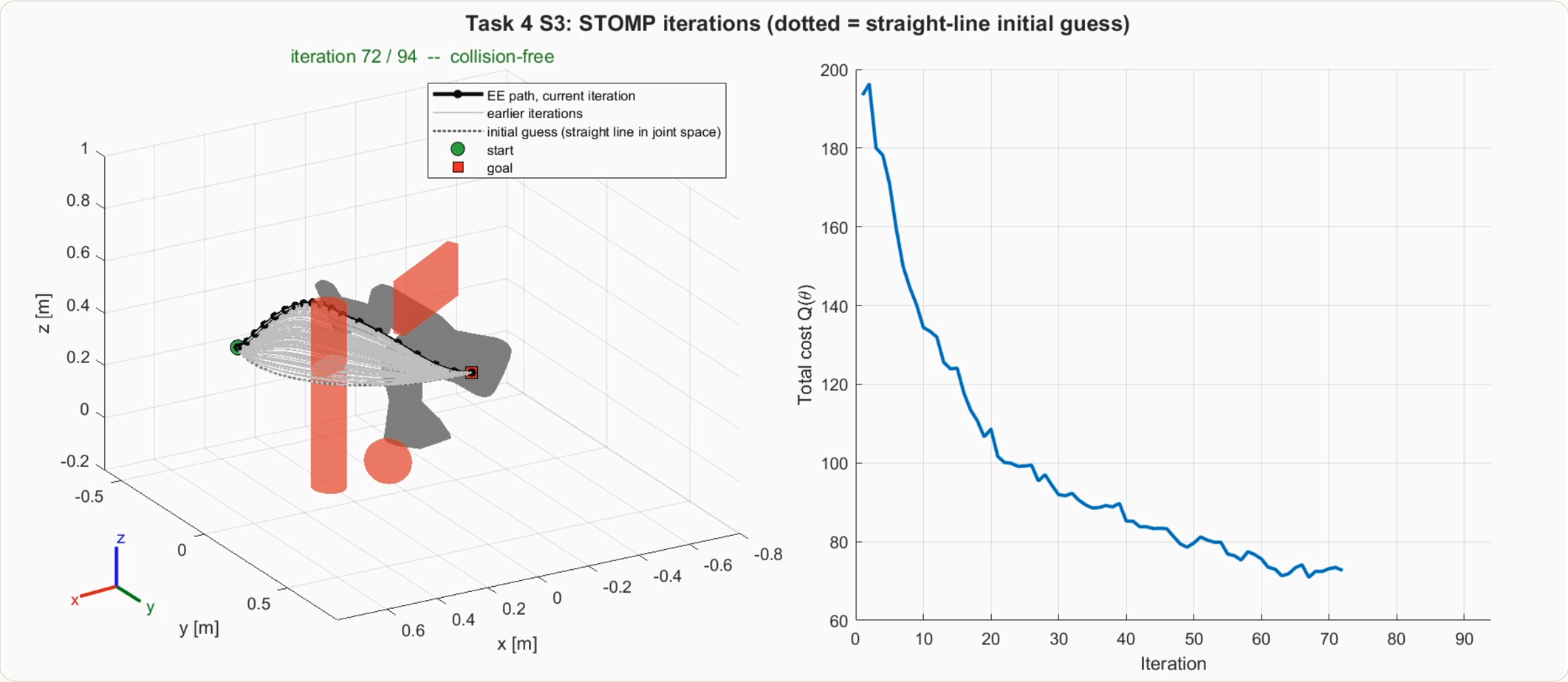


Around the column

5 of 20 waypoints collide at the start; 94 iterations

10 of 10 seeds collision-free in every scene. Boxes, rotated boxes, spheres and cylinders in one voxel world.

STOMP pushing the path off the column



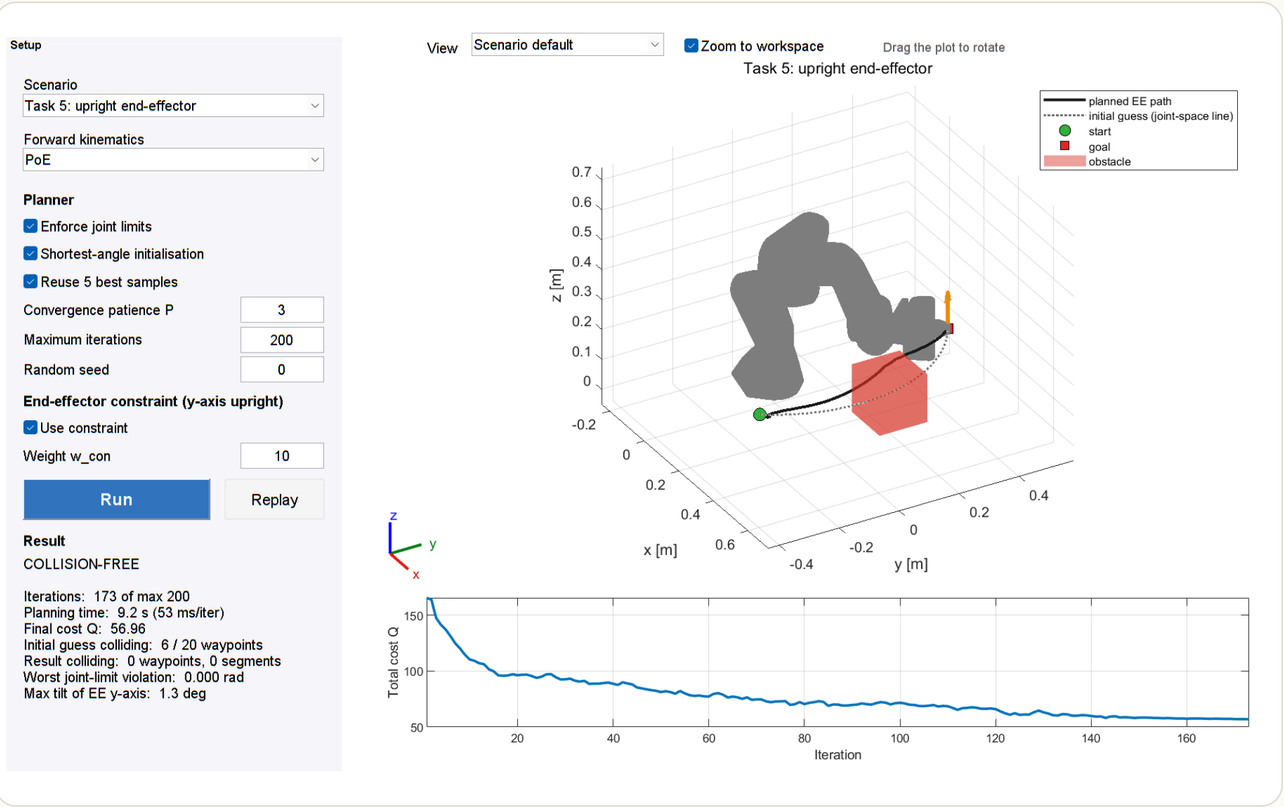
Extending STOMP: joint limits and sample reuse

Variant	Collision-free	Worst limit violation	S3 iterations
No extensions	35 / 35	0.52 rad	79
Joint limits	31 / 35	0	147
Limits + sample reuse (our default)	35 / 35	0	85
Limits + noise decay	29 / 35	0	78

Limits matter: plain STOMP left the Panda's limits by up to 30°, which a real arm cannot execute.

Reuse is free: the 5 cheapest samples are kept, with no extra kinematics. Noise decay made things worse, so it stays off.

Practical features



GUI

Pick a scene and options, plan, and rotate the view

±π joint angles

Shortest-angle start cuts a demo’s end-effector path from 3.31 m to 0.79 m

Ground truth

Mesh check at waypoints and 9 steps between each pair

Task 5: carrying a cup of coffee

Constraint cost at each waypoint (handout option 2)

$$q_c = \left\| \hat{y}_{sb} - \hat{z}_s \right\|_1$$

ŷ is the gripper's y-axis in the world frame, ẑ is world up. The cost is zero only when ŷ points straight up.

Cheap to compute

ŷ is a column of the rotation matrix FK already gives

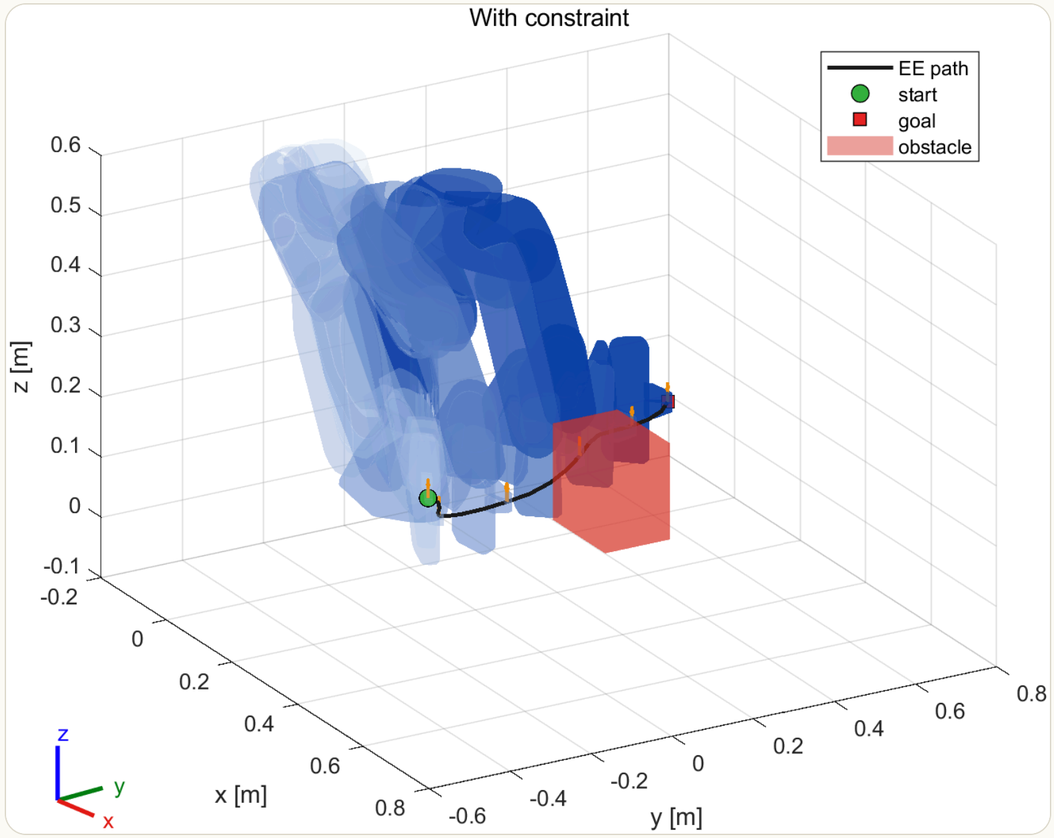
Added to S

Weighted by w_con, so upright samples get higher probability

One axis only

The gripper may still spin about the vertical; the cup stays upright

Task 5: designing an upright scene



Upright at both ends

0.5 m from the base axis, tool horizontal, y-axis vertical

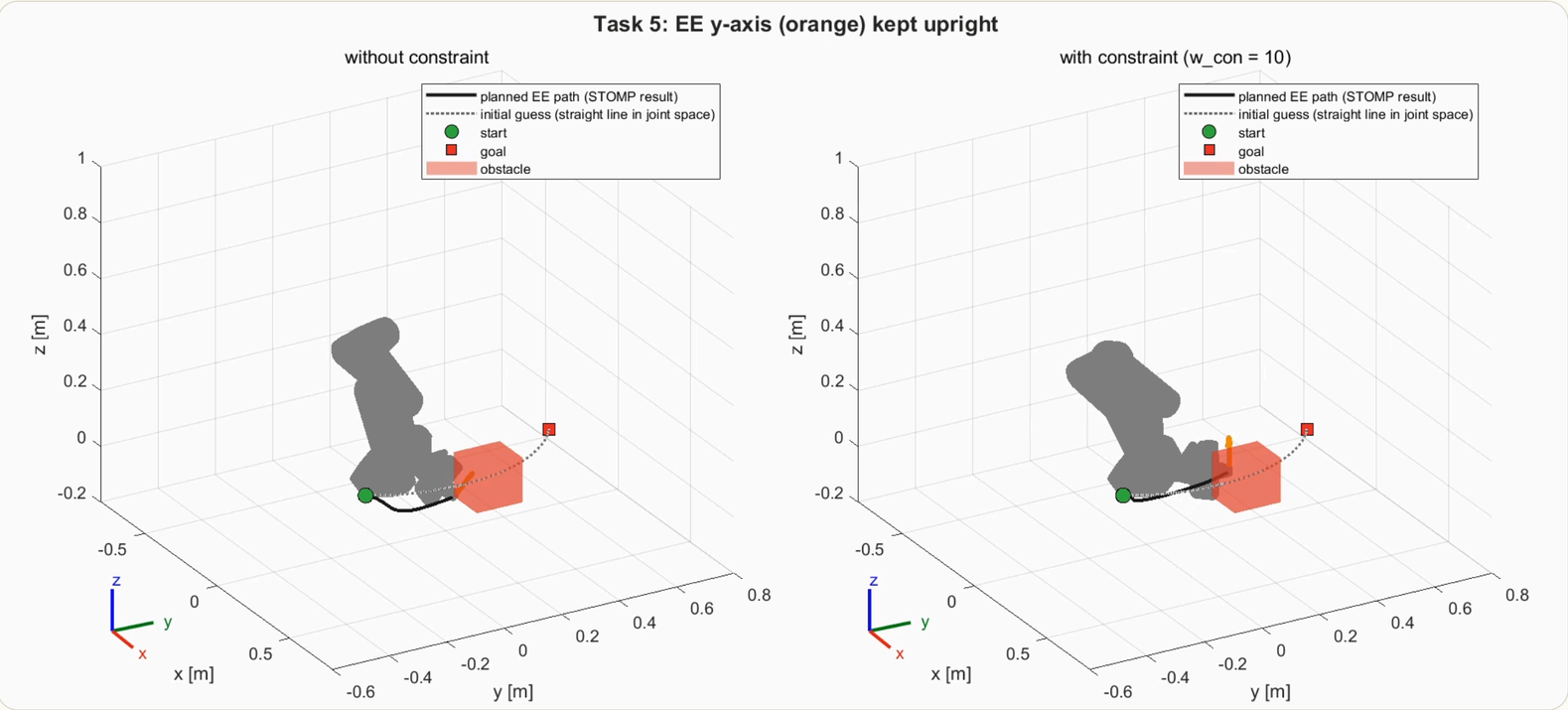
A box on the way

16 cm cube on the arc: 6 of 20 waypoints collide at the start

Height set by reachability

At 0.30 m IK stopped 2.8 cm off, joint 5 at its limit; at 0.20 m it is exact

Task 5: with and without the constraint



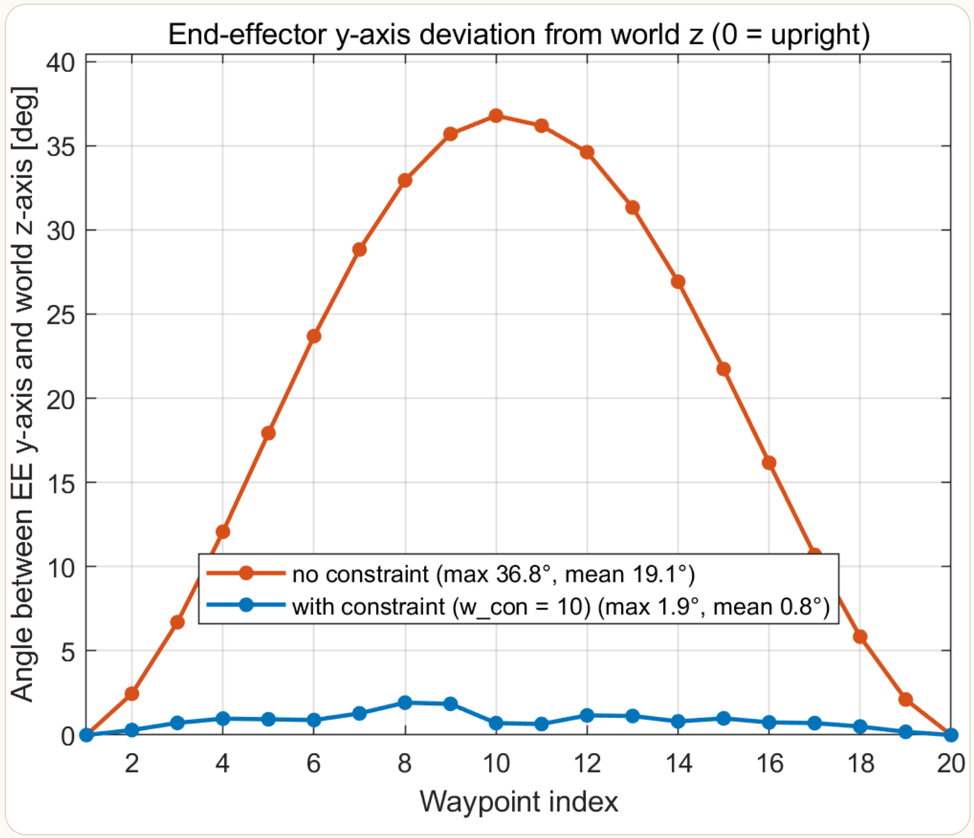
36.8° → 1.9°
maximum tilt

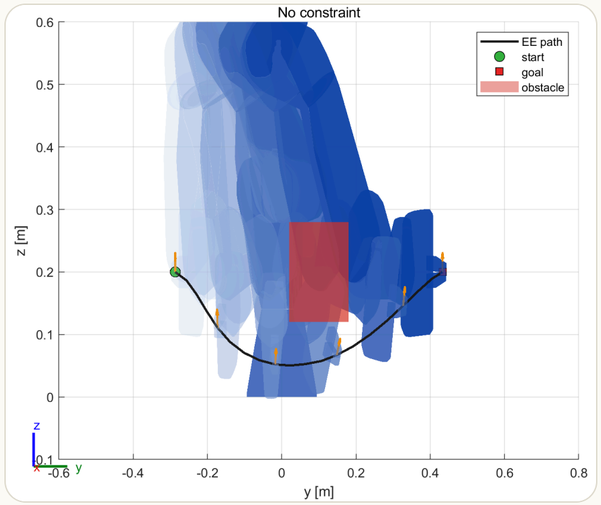
19.1° → 0.8°
mean tilt

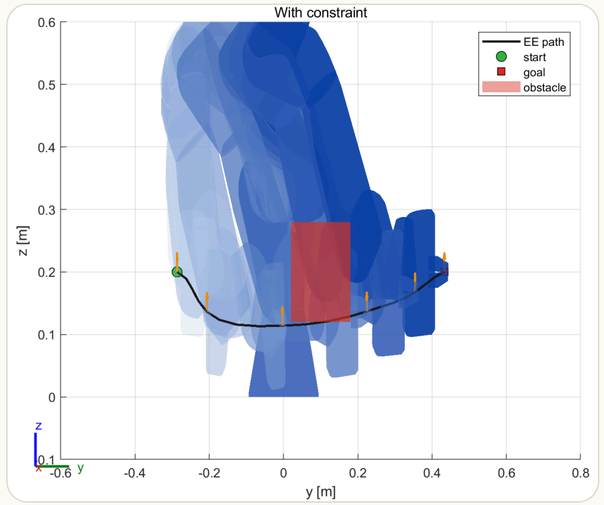
27 → 124
iterations; both collision-free

Orange arrow: the gripper's y-axis. It tilts on the left and stays vertical on the right; both plans avoid the box.

Task 5: tilt at every waypoint



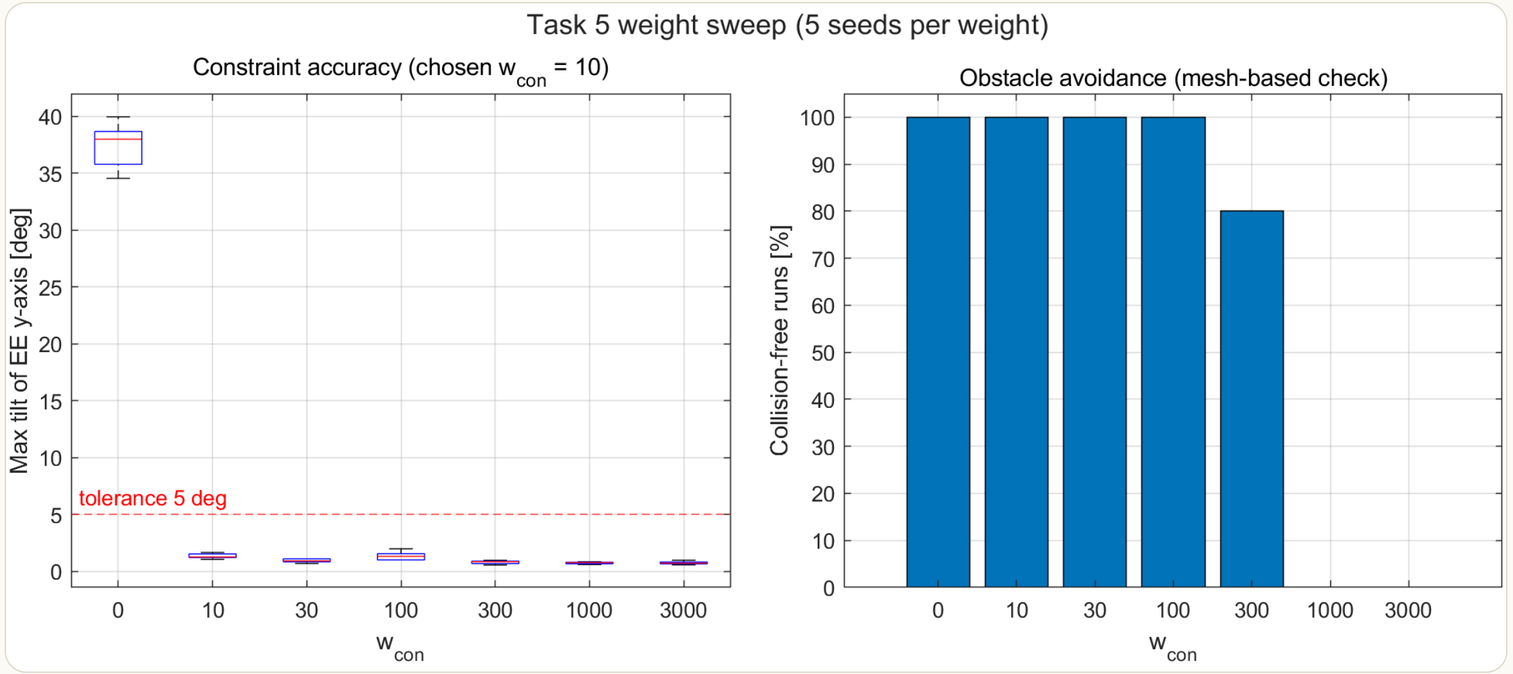




Similar path length: 0.839 m without, 0.821 m with the constraint.

Same cost per iteration: about 37 ms. The extra time comes from needing more iterations.

Task 5: choosing the constraint weight



$$w_{\text{con}} = 10$$

smallest weight that is collision-free on all 5 seeds, with tilt under 5°

Too strong fails

At 1000 and above, no run is collision-free: the constraint overrides avoidance

All five tasks solved and verified

What we added

- Joint limits and sample reuse
- Spheres fitted to the meshes
- Between-waypoint collision checks
- 10 sample-code bugs fixed; a GUI

Where the limits are

- Spheres hold 99% of the mesh, not all
- STOMP is local and depends on its start
- The upright constraint is soft: 1.9°, not 0°

Joshua: Tasks 1–2, framework, patience · **Jiamo:** Tasks 3–4, extensions, GUI · **Dexter:** Task 5, plots

AI (Claude) was used to aid the code, report and presentation.

Thank you. Questions?